

STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ
HÁLOVA 16, 851 01 BRATISLAVA

AI Travel Planner

Komplexná odborná maturitná práca

Bratislava

2025/26

Juraj Vépy

Ročník štúdia: 4.

STREDNÁ PRIEMYSELNÁ ŠKOLA ELEKTROTECHNICKÁ
HÁLOVA 16, 851 01 BRATISLAVA

AI Travel Planner

Komplexná odborná maturitná práca

Študijný odbor: 2573 M programovanie digitálnych technológií

Konzultant: Ing. Dominik Zatkalík, PhD.

Bratislava

2025/26

Juraj Vépy

Ročník štúdia: 4.

Čestné vyhlásenie

Vyhlasujem, že komplexnú odbornú maturitnú prácu na tému AI Travel Planner som vypracoval samostatne, s použitím uvedených literárnych zdrojov. Prácu som neprihlásil a ani neprezentoval v žiadnej inej súťaži, ktorá je pod gestorstvom MŠVVaM SR. Som si vedomý dôsledkov, ak uvedené údaje nie sú pravdivé.

V Bratislave, 23. 01. 2026

.....
Juraj Vépy

Pod'akovanie

Rád by som sa touto cestou poďakoval svojmu konzultantovi Ing. Dominikovi Zatkalíkovi, PhD. za prístup a odborné rady.

Abstrakt

Táto maturitná práca popisuje vývoj natívnej iOS aplikácie AI Travel Planner, ktorá využíva umelú inteligenciu na automatizáciu tvorby cestovných itinerárov. Práca analyzuje technologické rozdiely medzi natívnym vývojom v prostredí SwiftUI a multiplatformovými frameworkami, pričom na základe výkonnostných metrík zdôvodňuje voľbu jazyka Swift. Architektúra systému je postavená na modeli BFF (Backend-for-Frontend), ktorý pomocou Firebase Cloud Functions zabezpečuje bezpečnú komunikáciu s OpenAI API a ochranu API kľúčov. Implementované riešenie zahŕňa pokročilý state management pri viacstupňových formulároch, bezpečnú autentifikáciu cez protokoly OAuth 2.0 a asynchrónnu synchronizáciu dát s NoSQL databázou Firestore. Výsledkom je robustná mobilná aplikácia, ktorá transformuje neštruktúrované výstupy z LLM modelov do prehľadného a interaktívneho používateľského prostredia.

Kľúčové slová: iOS vývoj, SwiftUI, Swift, AI Travel Planner, Firebase, BFF architektúra, OAuth 2.0, OpenAI, NoSQL Firestore, MVVM.

Abstract

This thesis describes the development of a native iOS application, AI Travel Planner, which utilizes artificial intelligence to automate the creation of travel itineraries. The work analyzes the technological differences between native development in SwiftUI and cross-platform frameworks, justifying the choice of Swift based on performance metrics. The system architecture is built on the BFF (Backend-for-Frontend) model, using Firebase Cloud Functions to ensure secure communication with the OpenAI API and API key protection. The implemented solution includes advanced state management for multi-step forms, secure authentication via OAuth 2.0 protocols, and asynchronous data synchronization with the Firestore NoSQL database. The result is a robust mobile application that transforms unstructured outputs from LLM models into a clear and interactive user environment.

Keywords: iOS Development, SwiftUI, Swift, AI Travel Planner, Firebase, BFF Architecture, OAuth 2.0, OpenAI, NoSQL Firestore, MVVM.

Obsah

0	ÚVOD.....	8
1	Architektúra mobilnej aplikáci	9
1.1	Komunikačný model a Tok dát.....	9
1.2	Autentifikácia a bezpečnosť (OAuth).....	10
1.3	INTEGRÁCIA a spracovanie výstupov AI	10
2	Technologické možnosti vývoja	11
2.1	Porovnanie native a multiplatform vývoja.....	11
2.1.1	Vykresľovanie UI (rendering).....	11
2.2	Voľba vývojového prostredia: Swift a SwiftUI.....	12
2.3	Architektúra MVVM (Model-View-ViewModel).....	12
3	proces generovania plánu pomocou AI	13
3.1	Princíp fungovania LLM	13
3.2	Návrh štruktúry promptu	13
3.3	Integrácia AI API do backendu	14
4	Návrh a tvorba používateľského rozhrania.....	15
4.1	základy UI.....	15
4.2	Štruktúra UI vrstvy	15
4.3	Dizajnové princípy.....	16
4.4	Navigácia	16
5	Implementácia interaktívnych funkcií	17
5.1	State management v formulároch	17
5.2	Implementácia viacstupňového formulára.....	17
5.3	Interaktívne prvky a používateľské vstupy.....	18
5.4	Validácia a podmienené správanie	18
6	Návrh a implementácia používateľov	19
6.1	Architektúra autentifikačného systému	19
6.2	Implementácia autentifikácie.....	19
6.3	Bezpečnostné aspekty a ochrana dát.....	20
6.4	Správa relácií a session persistence	20
7	Vytvorenie databázového modelu	21
7.1	Koncepčný model dát	21
7.2	Štruktúra kolekcí a dokumentov	21

7.3	Vnorené štruktúry a hierarchia dát.....	22
7.4	Škálovateľnosť a budúce rozšírenia.....	22
8	Popis a testovanie kvality aplikácie	23
8.1	Definícia kľúčových metrík kvality	23
8.2	Metodológia testovania výkonu.....	24
8.3	Testovanie presnosti AI	24
8.4	Testovanie stability	25
8.5	Kontinuálne monitorovanie a optimalizácia	25
9	stratégia nasadenia aplikácie do App Store	26
9.1	Príprava aplikácie na kontrolu	26
9.2	Metadata a dokumentácia	27
9.3	Bezpečnostné a právne aspekty	27
9.4	Proces review a komunikácia	28
10	Návrh marketingovej stratégie.....	29
10.1	App Store Optimization (ASO) stratégia.....	29
10.2	Globálna expanzia a lokalizácia	30
10.3	Cenová stratégia a monetizácia.....	30
10.4	Marketing a propagácia	31
10.5	Budovanie komunity.....	31
11	Záver	32
12	Zoznam použitej literatúry	33

Zoznam tabuliek, grafov a ilustrácií

Obrázok 1 Architektúra Backend-for-Frontend (BFF)	9
Obrázok 2 Schéma architektonického vzoru MVVM.....	12
Tabuľka 1 Porovnanie výkonnostných metrík platforiem	11

Zoznam skratiek, značiek a symbolov

AI – (z angl. Artificial Intelligence) Umelá inteligencia, odbor informatiky zaoberajúci sa tvorbou systémov schopných simulovať ľudské intelektuálne procesy, ako je učenie a riešenie problémov.

API – (z angl. Application Programming Interface) Rozhranie pre programovanie aplikácií, súbor protokolov a nástrojov, ktoré umožňujú vzájomnú komunikáciu a výmenu dát medzi rôznymi softvérovými aplikáciami.

BFF – (z angl. Backend-for-Frontend) Architektonický vzor, kde sa vytvára špecifická serverová vrstva určená výhradne pre potreby konkrétneho klientskeho rozhrania, čím sa zvyšuje bezpečnosť a efektivita komunikácie.

CPU – (z angl. Central Processing Unit) Centrálna procesorová jednotka, hlavný hardvérový komponent počítača alebo mobilného zariadenia, ktorý vykonáva inštrukcie softvérového kódu.

FPS – (z angl. Frames Per Second) Počet snímok za sekundu, jednotka vyjadrujúca frekvenciu vykresľovania obrazu, ktorá priamo ovplyvňuje plynulosť používateľského rozhrania.

HTTP – (z angl. Hypertext Transfer Protocol) Protokol na prenos hypertextových dokumentov, základný komunikačný protokol webu a mobilných aplikácií využívaný na výmenu dát medzi klientom a serverom.

JSON – (z angl. JavaScript Object Notation) Ľahký formát na výmenu dát založený na textovej reprezentácii objektov, ktorý je ľahko čitateľný pre ľudí a jednoducho spracovateľný pre stroje.

LLM – (z angl. Large Language Model) Veľký jazykový model, typ umelej inteligencie trénovaný na obrovskom množstve textových dát, schopný generovať text, prekladať a riešiť komplexné lingvistické úlohy.

MVVM – (z angl. Model-View-ViewModel) Architektonický vzor softvérového inžinierstva, ktorý oddeľuje biznis logiku aplikácie od jej používateľského rozhrania, čo uľahčuje vývoj a testovanie.

0 ÚVOD

V súčasnej digitálnej dobe čelíme obrovskému množstvu informácií, ktoré ovplyvňujú spôsob organizácie nášho voľného času. Hoci internet ponúka nekonečné množstvo recenzií a máp, proces spájania týchto dát do jedného plánu ostáva pre používateľa časovo náročný. Tradičné metódy plánovania vyžadujú manuálne vyhľadávanie v rôznych zdrojoch, čo často vedie k zbytočnej frustrácii.

V tejto oblasti sa otvára veľký priestor pre umelú inteligenciu. Rozvoj veľkých jazykových modelov umožňuje automatizovať tvorbu personalizovaného obsahu s vysokou presnosťou. Cieľom mojej maturitnej práce je využiť tieto technológie a vytvoriť mobilnú aplikáciu AI Travel Planner pre systém iOS. Aplikácia funguje ako inteligentný sprievodca, ktorý na základe vstupov ako destinácia, rozpočet či záujmy dokáže v priebehu sekúnd vygenerovať detailný a logicky usporiadaný itinerár.

Táto práca dokumentuje celý proces vzniku aplikácie od teoretickej analýzy trhu až po finálnu implementáciu bezpečnej autentifikácie a interaktívnych funkcií. Výsledkom je funkčný produkt, ktorý demonštruje silu spojenia mobilného vývoja a umelej inteligencie v praktickom živote.

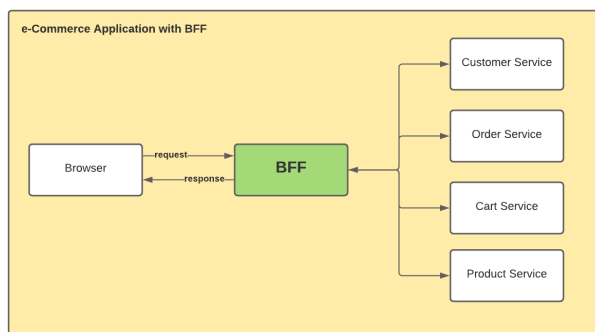
1 ARCHITEKTÚRA MOBILNEJ APLIKÁCII

Návrh architektúry moderných mobilných aplikácií vychádza z modelu Klient-Server, kde mobilná aplikácia (klient) slúži primárne ako prezentačná vrstva pre interakciu s používateľom, zatiaľ čo náročná logika a spracovanie dát prebiehajú na vzdialenom serveri (backend). V prípade aplikácie Travel Planner je architektúra rozšírená o integráciu s externým rozhraním umelej inteligencie (AI API). Celý systém sa skladá zo štyroch hlavných komponentov:

- **Mobilný klient (iOS App):** Zabezpečuje UI/UX, zbiera vstupy od používateľa a zobrazuje vygenerované plány.
- **Backend (Firestore/Cloud):** Slúži ako centrálny bod pre autentifikáciu používateľov a ukladanie dát.
- **Databáza (Firestore):** NoSQL databáza uchováajúca profily používateľov a históriu cestovných plánov.
- **AI Služba (OpenAI API):** Externá služba, ktorá na základe vstupov generuje textový a štruktúrovaný plán cesty.

1.1 KOMUNIKAČNÝ MODEL A TOK DÁT

Komunikácia medzi jednotlivými komponentmi prebieha prostredníctvom protokolu HTTP/REST. Mobilná aplikácia nekomunikuje s AI modelom priamo, ale požiadavky smeruje cez zabezpečený backend. Tento prístup, známy ako Backend-for-Frontend (BFF), zvyšuje bezpečnosť, keďže API kľúče k AI službe nie sú uložené priamo v zariadení používateľa.



Obrázok 1 Architektúra Backend-for-Frontend (BFF)

(Zdroj: <https://blog.bitsrc.io>, rok: 2021)

1.2 AUTENTIFIKÁCIA A BEZPEČNOSŤ (OAUTH)

Kľúčovou súčasťou architektúry je bezpečné overovanie totožnosti používateľov. Aplikácia využíva protokol OAuth 2.0, ktorý je dnes priemyselným štandardom pre autorizáciu a autentifikáciu v mobilných systémoch. Podľa odbornej literatúry je pri mobilných aplikáciách kritické správne implementovať výmenu prístupových tokenov, pretože mobilné prostredie neumožňuje bezpečné presmerovanie (browser redirection) rovnakým spôsobom ako webové prehliadače, čo často vedie k zraniteľnostiam implementácie OAuth protokolu. [1]

V mojej aplikácii preto využívam Firebase Authentication, ktoré zjednodušuje proces overovania OAuth tokov a poskytuje bezpečné prihlásenie. Tento systém zabezpečuje nielen registráciu a prihlásenie používateľov pomocou kombinácie emailu a hesla, ale aj integráciu s poskytovateľmi identít tretích strán, ako sú Google alebo Apple. Dôležitou súčasťou je automatizovaná správa používateľských relácií, ktorá zahŕňa bezpečné ukladanie a pravidelné obnovovanie prístupových tokenov (Refresh Tokens), vďaka čomu používateľ nemusí opakovane zadávať svoje prihlasovacie údaje pri každom spustení aplikácie. Z hľadiska bezpečnosti API volaní je každá požiadavka smerujúca na backend podpísaná v HTTP hlavičke pomocou tzv. Bearer Tokenu. Server tento token pred spracovaním akejkoľvek požiadavky overí, čím sa zabezpečí, že k dátam pristupuje len autorizovaný používateľ. Tento centralizovaný prístup eliminuje riziko ukladania citlivých údajov, ako sú heslá v otvorenom tvare, priamo v úložisku mobilného zariadenia a deleguje zodpovednosť za bezpečnosť na overenú a pravidelne auditovanú infraštruktúru poskytovateľa identity.)

1.3 INTEGRÁCIA A SPRACOVANIE VÝSTUPOV AI

Posledným kľúčovým prvkom architektúry je spôsob, akým aplikácia spracováva neštruktúrované výstupy z generatívnej umelej inteligencie. Keďže veľké jazykové modely (LLM) prirodzene generujú voľný text, pre potreby mobilnej aplikácie je nevyhnutné tento výstup transformovať do štruktúrovanej podoby. Architektúra preto zahŕňa vrstvu pre spracovanie dát (Data Parsing Layer) na strane backendu. [1]

Po prijatí odpovede od AI modelu backend analyzuje text a extrahuje z neho kľúčové entity, ako sú názvy pamiatok, časy návštev, odhadované ceny a geografické súradnice.

2 TECHNOLOGICKÉ MOŽNOSTI VÝVOJA

Pri návrhu mobilnej aplikácie Travel Planner bolo kľúčovým rozhodnutím zvolenie vhodného technologického stacku. V súčasnosti dominujú dva hlavné prístupy: natívny vývoj (Native) a multiplatformový vývoj (Cross-platform). Táto kapitola analyzuje tieto možnosti na základe aktuálnych výskumov a zdôvodňuje výber natívnej platformy iOS.

2.1 POROVNANIE NATÍVNEJ A MULTIPLATFORM VÝVOJA

Súčasný trh ponúka vývojárom možnosť písať aplikácie buď v jazykoch špecifických pre danú platformu (Swift pre iOS, Kotlin pre Android), alebo využiť frameworky ako Flutter či React Native, ktoré umožňujú nasadenie jedného kódu na viacero systémov. Hoci multiplatformové riešenia sľubujú rýchlejší vývoj, štúdie ukazujú značné rozdiely vo výkone a efektívnosti. Podľa porovnávacej štúdie Boyraza (2025), ktorá analyzovala identické aplikácie pre plánovanie trás vytvorené v Kotlin (Native) a Flutteri, dosahujú natívne aplikácie výrazne lepšie výsledky v kľúčových metrikách.

2.1.1 VYKRESĽOVANIE UI (RENDERING)

Natívna aplikácia dosahovala stabilných 55 FPS s mierou zasekávania (jank rate) iba 8,25 %. Naopak, aplikácia vo Flutteri mala pri náročných operáciách (skrolovanie zoznamu, zoomovanie mapy) priemernú rýchlosť len 33 FPS a mieru zasekávania až 55,69 %, čo viedlo k viditeľne trhanejšiemu obrazu. [2]

Metrika	Natívny vývoj	Multiplatform
Snímková frekvencia	55 FPS	33 FPS
Miera zasekávania (Jank)	8,25 %	55,69 %
Záťaž CPU (Idle stav)	3,7 %	17,8 %
Zmena pamäte (RAM)	-40 MB (uvoľnenie)	+28 MB (nárast)

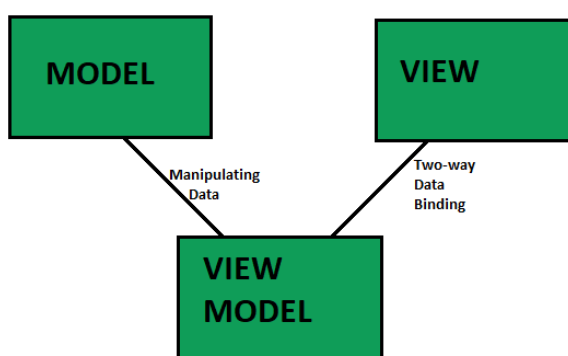
Tabuľka 1 Porovnanie výkonnostných metrik platform

2.2 VOĽBA VÝVOJOVÉHO PROSTREDIA: SWIFT A SWIFTUI

Na základe analýzy výkonu som pre implementáciu zvolil natívny vývoj pre platformu iOS. Ako hlavný programovací jazyk bol použitý Swift, ktorý je moderným, typovo bezpečným a kompilovaným jazykom priamo od spoločnosti Apple. Pre tvorbu používateľského rozhrania som využil framework SwiftUI. Ide o deklaratívny UI framework, ktorý predstavuje zásadnú zmenu paradigmy oproti staršiemu imperatívnemu prístupu (UIKit). Podľa štúdie Junga a Seovej (2023) prináša deklaratívny prístup v SwiftUI niekoľko kľúčových výhod, ktoré zásadne ovplyvňujú rýchlosť a kvalitu vývoja. [3] Vývojár definuje UI a systém sa automaticky postará o jeho vykreslenie, čím sa eliminuje nutnosť manuálnej synchronizácie medzi dátami a zobrazením. Vďaka integrácii s frameworkom Combine umožňuje SwiftUI efektívne reagovať na asynchrónne udalosti a zmeny stavu v reálnom čase. Deklaratívny kód je čitateľnejší a menej náchylný na chyby, čo je pri komplexných aplikáciách s dynamickým obsahom kľúčové.

2.3 ARCHITEKTÚRA MVVM (MODEL-VIEW-VIEWMODEL)

Aby sme zabezpečili oddelenie logiky od prezentácie, aplikácia je postavená na architektonickom vzore MVVM. Tento vzor je pre SwiftUI prirodzený, pretože využíva reaktívne prepojenie dát, ktoré je v modernom iOS vývoji štandardom. [3]



Obrázok 2 Schéma architektonického vzoru MVVM

(Zdroj: www.geeksforgeeks.org, rok: 2023)

3 PROCES GENEROVANIA PLÁNU POMOCOUI AI

Jadrom aplikácie Travel Planner je schopnosť automaticky vygenerovať personalizovaný plán na základe vstupných parametrov používateľa. Tento proces využíva pokročilé schopnosti veľkých jazykových modelov (LLM), konkrétne modelu GPT od spoločnosti OpenAI, prístupného cez API rozhranie.

3.1 PRINCÍP FUNGOVANIA LLM

Veľké jazykové modely nefungujú ako tradičné databázové vyhľadávače, ktoré len vrátia uložené informácie. Namiesto toho generujú text predikciou nasledujúcich slov na základe kontextu a naučených vzorcov z obrovského množstva tréningových dát. To im umožňuje vytvárať kreatívne a unikátne plány ciest, ktoré kombinujú logické následnosti s geografickými znalosťami. Pre efektívne využitie tohto modelu je kľúčová technika zvaná Prompt Engineering. Ide o proces navrhovania presných inštrukcií (promptov), ktoré modelu jasne definujú, čo má urobiť. Podľa oficiálnej dokumentácie OpenAI je kvalita výstupu priamo úmerná kvalite a špecifickosti vstupu. Model nedokáže čítať myšlienky, a preto je nutné explicitne definovať formát, dĺžku a štýl odpovede, aby sa predišlo generovaniu irelevantných alebo príliš všeobecných informácií. [4]

3.2 NÁVRH ŠTRUKTÚRY PROMPTU

Vo svojej aplikácii som navrhol a implementoval komplexný systémový prompt, ktorý slúži ako most medzi neštruktúrovanými požiadavkami používateľa (ako sú cieľová destinácia, rozpočet či osobné záujmy) a presne definovaným dátovým výstupom, ktorý vyžaduje mobilná aplikácia. Pri konštrukcii tohto promptu som dôsledne aplikoval metodiku *Prompt Engineeringu* vychádzajúcu z odporúčaní pre optimalizáciu veľkých jazykových modelov. [4] Základom môjho prístupu je technika pridelenia role (Persona Adoption), kedy modelu hneď v úvode definujem jeho pozíciu experta. Inštrukciou zabezpečujem, že model prispôsobí tón a odbornosť generovanej odpovede očakávanej kvalite profesionálneho sprievodcu. Následne v texte promptu definujem explicitné obmedzenia, ktorých cieľom je eliminovať zbytočný text. Vďaka tomu redukujem pravdepodobnosť generovania všeobecných fráz, čím šetrím tokeny a zvyšujem informačnú hustotu výstupu. Najkritickejšou časťou promptu je však presná definícia výstupného formátu. Keďže backend aplikácie musí odpoveď strojovo spracovať, nemôžem sa spoliehať na voľný text. Preto vyžadujem striktný JSON formát

a využívam techniku *Few-shot prompting*, kde modelu priamo v zadaní poskytujem vzor požadovanej štruktúry. Tento prístup minimalizuje chyby pri parsovaní a zabezpečuje konzistentnú štruktúru dát pre zobrazenie v aplikácii.

3.3 INTEGRÁCIA AI API DO BACKENDU

Po navrhnutí optimálneho promptu je ďalším kritickým krokom technická implementácia komunikácie medzi backendom a OpenAI API. V mojej aplikácii som zvolil architektúru, kde mobilný klient nekomunikuje s AI službou priamo, ale všetky požiadavky prechádzajú cez vlastný backend server postavený na Firebase Cloud Functions. Tento prístup prináša niekoľko kľúčových výhod z hľadiska bezpečnosti, nákladov a kontroly nad dátovým tokom.

Backend server slúži ako zabezpečený proxy medzi mobilnou aplikáciou a OpenAI API. Keď používateľ zadá požiadavku na generovanie plánu, mobilná aplikácia odošle HTTP POST požiadavku na môj backend endpoint, ktorý obsahuje všetky potrebné parametre (destinácia, dátum, rozpočet, preferencie). Backend následne sformuluje finálny prompt kombináciou systémovej inštrukcie a používateľských vstupov, pričom automaticky pridá kontextové informácie, ako je aktuálny dátum (pre správne plánovanie sezónnych aktivít) alebo geografické obmedzenia.

Samotné volanie OpenAI API prebieha prostredníctvom oficiálneho REST API rozhrania, ktoré využíva protokol HTTPS a autentifikáciu pomocou API kľúča uloženého bezpečne na serveri. Požiadavka je štruktúrovaná ako JSON objekt obsahujúci pole správ, kde prvá správa definuje systémovú rolu a druhá obsahuje vlastný prompt s používateľskými požiadavkami. Backend špecifikuje aj technické parametre volania, ako je názov modelu (v mojom prípade GPT-4o-mini pre optimálny pomer cena/výkon), maximálna dĺžka odpovede (`max_tokens`) a teplota (`temperature`), ktorá ovplyvňuje kreativitu generovania. [4]

Po prijatí odpovede od OpenAI API backend vykonáva dvojfázové spracovanie. V prvej fáze sa overuje syntaktická správnosť JSON štruktúry pomocou štandardného parsera. Ak parsovanie zlyhá, systém sa pokúsi automaticky opraviť bežné chyby, ako sú neuzavreté zátvorky alebo chýbajúce úvodzovky, čo je častý problém pri generovaní štruktúrovaných dát jazykovými modelmi. V druhej fáze sa vykonáva sémantická validácia, kde sa kontroluje, či vygenerované dáta spĺňajú očakávanú schému (napr. či každý deň obsahuje pole aktivít, či sú časy v správnom formáte a či sú súradnice v platnom rozsahu).

4 NÁVRH A TVORBA POUŽÍVATEĽSKÉHO ROZHRAINIA

Používateľské rozhranie mobilnej aplikácie je prvým kontaktným bodom s používateľom a zásadne ovplyvňuje celkový dojem a úspešnosť produktu. Pri návrhu Travel Planner som sa riadil princípmi moderného iOS dizajnu a využil som výhody deklaratívneho prístupu SwiftUI, ktorý umožňuje efektívne vytvárať responzívne a intuitívne rozhrania.

4.1 ZÁKLADY UI

Rozhranie aplikácie je postavené na architektúre MVVM, ktorá je prirodzene kompatibilná so SwiftUI vďaka svojmu stavovo-riadenému modelu. Podľa výskumu Moloudiho (2025) je MVVM v kontexte SwiftUI aplikácií vhodný pre rýchly vývoj a prirodzenú integráciu s deklaratívnym paradigom, kde View automaticky reaguje na zmeny stavu vystavené cez ViewModel. [5] Tento prístup eliminuje potrebu manuálnej synchronizácie medzi dátami a UI, čo je kľúčové pre udržanie konzistentného používateľského zážitku.

V mojej implementácii každý SwiftUI View pozoruje príslušný ViewModel pomocou property wrapperov `@ObservedObject` alebo `@StateObject`, čím sa zabezpečuje, že akékoľvek zmeny v dátach (napr. načítanie nového cestovného plánu alebo aktualizácia preferencií) sa okamžite prejavia v používateľskom rozhraní bez nutnosti explicitného volania metód na obnovenie UI. Táto reaktívna povaha SwiftUI v kombinácii s MVVM architektúrou vytvára robustný základ pre dynamické a responzívne rozhranie. Štúdie potvrdzujú, že MVVM v SwiftUI aplikáciách poskytuje prirodzený jednosmerný tok dát (unidirectional data flow), ktorý je v súlade so stavovo-riadenými aktualizáciami SwiftUI, čo výrazne zjednodušuje správu komplexných UI stavov. [5]

4.2 ŠTRUKTÚRA UI VRSTVY

Jedným z kľúčových aspektov návrhu UI vrstvy je zachovanie čistej separácie medzi prezentačnou logikou a biznis logikou. V mojej aplikácii sú SwiftUI Views navrhnuté tak, aby obsahovali výlučne deklaratívne definície rozhrania, pričom všetka logika súvisiaca so spracovaním dát, validáciou vstupov alebo komunikáciou so službami je delegovaná do ViewModel vrstvy. Tento prístup nie je len teoreticky správny, ale má aj praktické výhody z hľadiska testovateľnosti. Podľa porovnávacích štúdií architektúr

iOS aplikácií umožňuje MVVM efektívne testovanie UI logiky prostredníctvom izolovaných testov ViewModelov, kde sa môžu simulovať rôzne stavy a overovať správanie bez nutnosti spúšťania celého UI. [5]

4.3 DIZAJNOVÉ PRINCÍPY

Pri tvorbe aplikácie som sa riadil zásadou minimalizmu a čitateľnosti, ktoré sú kľúčové pre úspešné mobilné aplikácie. Hlavná obrazovka (HomeView) využíva čistý layout s jasnou hierarchiou informácií: uvítacia správa s menom používateľa v hornej časti, centrálné umiestnený call-to-action tlačidlo pre vytvorenie nového plánu a jednoduchá navigácia cez toolbar menu. Tento prístup je v súlade s odporúčaniami pre mobilný UX dizajn, ktoré zdôrazňujú, že aplikácia by mala poskytovať používateľovi jasné cesty a funkcionality na dokončenie prioritných úloh, pričom len primárny obsah a funkcionality by mali byť viditeľné štandardne, zatiaľ čo sekundárny obsah by mal byť skrytý, ale dostupný cez menu alebo gestá. [6]

Farbová schéma je založená na systémových farbách iOS, čo zabezpečuje konzistentnosť s platformou a automatickú podporu pre tmavý režim (Dark Mode). V kontexte brand identity som aplikoval princíp, že logá a brand elementy by mali byť použité jemne a šetrne, pretože priestor obrazovky mobilných aplikácií je obmedzený a používatelia už vynaložili úsilie na stiahnutie aplikácie, takže poznajú značku.

4.4 NAVIGÁCIA

Navigačná štruktúra aplikácie je navrhnutá tak, aby bola intuitívna a umožňovala používateľovi rýchlo pristupovať k hlavným funkciám. Aplikácia využíva hierarchickú navigáciu založenú na NavigationStack, ktorá umožňuje prechádzať medzi obrazovkami prirodzeným gestom "swipe back", čo je štandardné správanie v iOS aplikáciách. Hlavná obrazovka slúži ako centrálné miesto, odkiaľ používateľ môže vytvoriť nový plán, pristúpiť k nastaveniam alebo odhlásiť sa.

Podľa best practices pre mobilný UX dizajn by navigácia mala byť jasná, orientovaná na úlohy a logická, pričom ovládacie prvky obrazovky by mali naznačovať, ako ich používať. [6] Toolbar menu v pravom hornom rohu poskytuje rýchly prístup k sekundárnym funkciám, ako sú profil a nastavenia, pričom zachováva čistý vzhľad hlavnej obrazovky. Tento prístup minimalizuje kognitívne zaťaženie používateľa a umožňuje mu sa sústrediť na plánovanie cesty.

5 IMPLEMENTÁCIA INTERAKTÍVNYCH FUNKCIÍ

Interaktívne funkcie aplikácie Travel Planner sú postavené na princípoch SwiftUI state managementu, kde sa používateľské rozhranie automaticky aktualizuje pri zmene stavu aplikácie. Tento prístup eliminuje potrebu manuálnej synchronizácie medzi dátami a UI, čo je kľúčové pre vytvorenie plynulého a intuitívneho používateľského zážitku. [7]

5.1 STATE MANAGEMENT V FORMULÁROCH

Jadrom interaktívnych funkcií aplikácie je správne použitie property wrapperov SwiftUI, ktoré zabezpečujú reaktívne prepojenie medzi používateľskými vstupmi a stavom aplikácie. V mojej implementácii formulára na vytvorenie cestovného plánu využívam state management pre lokálne hodnoty, ktoré sa menia v rámci jedného view, a mechanizmy pre komunikáciu medzi parent a child views. [7]

Formulár je rozdelený do štyroch etáp, pričom každá etapa má vlastné state premenné, ktoré uchovávajú aktuálne hodnoty. Keď používateľ zmení hodnotu v textovom poli alebo vyberie dátum, SwiftUI automaticky detekuje zmenu a prekreslí príslušnú časť UI, čím sa zabezpečí, že zobrazenie vždy zodpovedá aktuálnemu stavu. Tento mechanizmus funguje transparentne bez nutnosti explicitného volania metód na obnovenie rozhrania, čo výrazne zjednodušuje vývoj a redukuje možnosť chýb. [7]

5.2 IMPLEMENTÁCIA VIACSTUPŇOVÉHO FORMULÁRA

Viacstupňový formulár je implementovaný pomocou page-based navigácie, ktorá umožňuje plynulé prechody medzi jednotlivými etapami. Každá etapa je reprezentovaná samostatným view komponentom, ktoré komunikujú s parent view prostredníctvom binding mechanizmov. Tento prístup umožňuje, aby každá etapa mohla modifikovať hodnoty v parent view, pričom parent view si zachováva kontrolu nad celkovým stavom formulára a validáciou vstupov. Kľúčovým aspektom implementácie je správna voľba klávesnice. Keď používateľ prechádza medzi etapami, klávesnica sa automaticky skryje, čo zabezpečuje plynulý používateľský zážitok bez rušivých prechodov. Animácie medzi etapami sú implementované pomocou SwiftUI animačného systému, čo vytvára vizuálne príjemný prechod pri zmene etapy. Progress bar v hornej časti formulára je reaktívne prepojený so stavom, takže pri každej zmene etapy sa automaticky aktualizuje vizuálna reprezentácia pokroku používateľa.

5.3 INTERAKTÍVNE PRVKY A POUŽÍVATELSKÉ VSTUPY

Aplikácia obsahuje niekoľko typov interaktívnych prvkov, z ktorých každý využíva špecifické SwiftUI property wrappers pre správu stavu. Date picker v druhej etape používa binding pre prepojenie s dátumovými premennými v parent view. Keď používateľ vyberie dátum, zmena sa automaticky prejaví v parent view bez nutnosti explicitného callbacku, čo je jednou z hlavných výhod deklaratívneho prístupu SwiftUI. [7]

Budget selection v tretej etape využíva tri tlačidlá reprezentujúce rôzne cenové kategórie. Keď používateľ klikne na tlačidlo, SwiftUI automaticky deteguje zmenu v state premennej a prekreslí UI, čím sa zvýrazní vybrané tlačidlo a zmení sa vzhľad ostatných. Tento prístup eliminuje potrebu manuálnej synchronizácie medzi stavom a zobrazením, čo je typické pre imperatívne frameworky, kde musí vývojár explicitne volať metódy na aktualizáciu UI pri každej zmene dát.

5.4 VALIDÁCIA A PODMIENENÉ SPRÁVANIE

Dôležitou súčasťou interaktívnych funkcií je validácia vstupov a podmienené správanie UI. V mojej implementácii je tlačidlo na pokračovanie deaktivované, ak nie sú splnené podmienky pre pokračovanie v ďalšej etape. Toto je implementované pomocou computed property, ktorá kontroluje aktuálny stav formulára a vracia boolean hodnotu. Tlačidlo následne používa modifier pre deaktiváciu a zmenšenie opacity pre vizuálne indikovanie stavu.

Tento prístup je v súlade s princípom, že v SwiftUI je view funkciou stavu – ak sa zmení stav, view sa automaticky prekreslí a UI sa aktualizuje podľa nových podmienok. [7] Táto automatická synchronizácia eliminuje celú kategóriu chýb súvisiacich s nekonzistentným stavom UI, ktoré sú bežné v imperatívnych frameworkoch, kde musí vývojár manuálne zabezpečiť, aby sa UI aktualizovalo pri každej zmene stavu.

6 NÁVRH A IMPLEMENTÁCIA POUŽÍVATEĽOV

Bezpečná autentifikácia používateľov je základom každej mobilnej aplikácie, ktorá pracuje s osobnými dátami. V aplikácii Travel Planner som implementoval autentifikačný systém založený na Firebase Authentication, ktorý poskytuje robustné riešenie pre overovanie totožnosti používateľov a zabezpečenie prístupu k ich dátam. Tento systém je doplnený o Firebase Security Rules, ktoré definujú server-enforced pravidlá na ochranu dát v databáze a storage, čím sa zabezpečuje, že používatelia majú prístup len k vlastným dátam. [8]

6.1 ARCHITEKTÚRA AUTENTIFIKAČNÉHO SYSTÉMU

Autentifikačný systém je navrhnutý v súlade s MVVM architektúrou aplikácie, kde AuthenticationService slúži ako vrstva abstrakcie nad Firebase Authentication API a AuthViewModel spravuje stav autentifikácie a poskytuje reaktívne rozhranie pre UI vrstvu. Tento prístup umožňuje oddeliť logiku autentifikácie od prezentácie a zabezpečuje, že zmeny v autentifikačnom stave sa automaticky prejavajú v používateľskom rozhraní prostredníctvom SwiftUI property wrapperov.

Firebase Authentication poskytuje službu pre správu používateľských relácií, ktorá automaticky spravuje tokeny, refresh tokeny a session persistence. Keď sa používateľ prihlási, Firebase vygeneruje autentifikačný token, ktorý sa používa na overenie identity pri každej požiadavke na backend. Tento token je bezpečne uložený v systéme a automaticky sa obnovuje, keď vyprší jeho platnosť, čím sa zabezpečuje plynulý používateľský zážitok bez nutnosti opakovaného prihlasovania.

6.2 IMPLEMENTÁCIA AUTENTIFIKÁCIE

Aplikácia podporuje dva hlavné spôsoby prihlasovania: Apple Sign-In a Google Sign-In. Oba tieto prístupy využívajú OAuth 2.0 protokol, ktorý je priemyselným štandardom pre bezpečnú autentifikáciu. Implementácia Apple Sign-In využíva natívny iOS framework, ktorý poskytuje bezpečný spôsob overenia totožnosti pomocou Face ID, Touch ID alebo Apple ID hesla. Google Sign-In je implementovaný prostredníctvom oficiálneho SDK, ktorý zabezpečuje konzistentný zážitok naprieč platformami.

Po úspešnom prihlásení sa používateľské údaje automaticky synchronizujú s Firestore databázou, kde sa vytvorí alebo aktualizuje dokument používateľa. Tento dokument obsahuje základné informácie, ako sú email, meno a dátum registrácie, ktoré sa následne používajú v aplikácii na personalizáciu používateľského zážitku. Firebase

Security Rules zabezpečujú, že každý používateľ má prístup len k svojim vlastným dátam, pričom pravidlá sú definované na serveri a nie v klientskej aplikácii, čo eliminuje možnosť obchádzania bezpečnostných kontrol. [8]

6.3 BEZPEČNOSTNÉ ASPEKTY A OCHRANA DÁT

Kritickým aspektom autentifikačného systému je zabezpečenie, že dáta používateľov sú chránené pred neoprávneným prístupom. Firebase Security Rules poskytujú flexibilný a rozšíriteľný konfiguračný jazyk na definovanie, aké dáta môžu používatelia pristupovať. Tieto pravidlá fungujú tak, že porovnávajú vzor s cestami v databáze a následne aplikujú vlastné podmienky na povolenie prístupu k dátam na týchto cestách. [8]

V mojej implementácii sú Security Rules navrhnuté tak, aby každý používateľ mal prístup len k svojim vlastným dokumentom v kolekcii používateľov a cestovných plánov. Pravidlá využívajú autentifikačný kontext z Firebase Authentication na overenie identity používateľa a porovnanie jeho ID s ID vlastníka dokumentu. Tento prístup zabezpečuje, že aj keby mal útočník prístup k autentifikačnému tokenu, nemôže pristupovať k dátam iných používateľov, pretože pravidlá sú vynucované na serveri a nie v klientskej aplikácii. [8]

6.4 SPRÁVA RELÁCIÍ A SESSION PERSISTENCE

Jednou z kľúčových výhod Firebase Authentication je automatická správa používateľských relácií. Keď sa používateľ prihlási, Firebase automaticky uloží autentifikačný token a refresh token, ktoré sa používajú na udržanie prihlásenej relácie aj po reštarte aplikácie. Tento mechanizmus zabezpečuje, že používateľ nemusí opakovane zadávať svoje prihlasovacie údaje pri každom spustení aplikácie, čo výrazne zlepšuje používateľský zážitok.

V mojej implementácii AuthViewModel automaticky kontroluje autentifikačný stav pri inicializácii a načítava používateľské údaje z Firestore, ak je používateľ stále prihlásený. Tento prístup zabezpečuje, že aplikácia vždy má aktuálne informácie o používateľovi a môže správne zobrazovať personalizovaný obsah. Ak používateľ nie je prihlásený alebo jeho relácia vypršala, aplikácia automaticky presmeruje na prihlasovaciu obrazovku.

7 VYTVORENIE DATABÁZOVÉHO MODELU

Návrh databázového modelu pre mobilnú aplikáciu vyžaduje iný prístup než pri tradičných relačných databázach. Firestore, ktorý používam v aplikácii Travel Planner, je dokumentovo orientovaná NoSQL databáza, ktorá nevyžaduje preddefinovanú schému, ale správny návrh kolekcí a dokumentov je kľúčový pre výkon a škálovateľnosť aplikácie. Podľa výskumu Miora a kol. (2016) je návrh schémy pre NoSQL databázy kritický pre vysoký výkon, pričom výber vhodnej štruktúry úzko súvisí s očakávaným workloadom aplikácie. [9]

7.1 KONCEPČNÝ MODEL DÁT

Databázový model aplikácie Travel Planner je založený na dvoch hlavných entitách: používateľoch a cestovných plánoch. Používateľská entita (User) obsahuje základné informácie o používateľovi, ako sú identifikátor, email, meno a dátum registrácie. Cestovný plán (TravelPlan) je komplexnejšia entita, ktorá obsahuje informácie o destinácii, dátumoch cesty, rozpočte a detailný itinerár rozčlenený po dňoch.

Kľúčovým rozhodnutím pri návrhu modelu bolo, ako reprezentovať hierarchickú štruktúru cestovného plánu, ktorá obsahuje dni, aktivity a reštaurácie. V relačnej databáze by som použil normalizovaný prístup s viacerými tabuľkami a foreign key vzťahmi. V NoSQL databáze som však zvolil denormalizovaný prístup, kde celý cestovný plán vrátane všetkých dní, aktivít a reštaurácií je uložený ako jeden dokument v kolekcii travelPlans. Tento prístup je v súlade s best practices pre NoSQL databázy, ktoré odporúčajú denormalizovať dáta v prospech rýchlejšieho čítania, keďže väčšina operácií v aplikácii zahŕňa načítanie celého plánu naraz, namiesto častí. [9]

7.2 ŠTRUKTÚRA KOLEKCIÍ A DOKUMENTOV

Databáza je organizovaná do dvoch hlavných kolekcí: users a travelPlans. Kolekcia users obsahuje jeden dokument pre každého používateľa, kde dokument ID zodpovedá Firebase Authentication UID, čo zabezpečuje jedinečnosť a umožňuje jednoduché vyhľadávanie. Každý dokument používateľa obsahuje základné informácie potrebné pre personalizáciu aplikácie. Kolekcia travelPlans obsahuje dokumenty reprezentujúce jednotlivé cestovné plány.

Každý dokument obsahuje pole userId, ktoré identifikuje vlastníka plánu a umožňuje efektívne dotazy na všetky plány konkrétného používateľa. Táto štruktúra

je optimalizovaná pre najčastejšie operácie v aplikácii: vytvorenie nového plánu, načítanie všetkých plánov používateľa a načítanie konkrétneho plánu podľa ID. Podľa výskumu o NoSQL schema designe by schéma mala reflektovať očakávaný workload, pričom štruktúra dát by mala umožňovať odpovedať na najčastejšie dotazy s minimálnym počtom databázových operácií. [9]

7.3 VNORENÉ ŠTRUKTÚRY A HIERARCHIA DÁT

Cestovný plán obsahuje pole days, ktoré je pole objektov reprezentujúcich jednotlivé dni itinerára. Táto štruktúra je uložená ako vnorené objekty v jednom dokumente, čo eliminuje potrebu join operácií a umožňuje načítanie celého plánu jednou databázovou operáciou. Tento prístup má výhody aj nevýhody. Hlavnou výhodou je rýchlosť čítania, načítanie celého plánu vyžaduje len jednu operáciu get, čo je kritické pre mobilné aplikácie, kde latencia siete môže výrazne ovplyvniť používateľský zážitok. Nevýhodou je, že ak by som potreboval upraviť len jednu aktivitu v jednom dni, musím aktualizovať celý dokument plánu, čo môže byť neefektívne pri veľkých dokumentoch. V kontexte mojej aplikácie však plány nie sú tak rozsiahle, aby to predstavovalo problém, a väčšina operácií zahŕňa prácu s celým plánom naraz. [9]

7.4 ŠKÁLOVATEĽNOSŤ A BUDÚCE ROZŠÍRENIA

Návrh databázového modelu berie do úvahy potrebu škálovateľnosti. Keďže každý cestovný plán je uložený ako samostatný dokument, databáza môže efektívne škálovať horizontálne, pričom každý dokument môže byť uložený na rôznych serveroch bez ovplyvnenia výkonu. Tento prístup je typický pre NoSQL databázy, ktoré sú navrhnuté pre horizontálne škálovanie, na rozdiel od relačných databáz, ktoré často vyžadujú vertikálne škálovanie. Pre budúce rozšírenia, ako je zdieľanie plánov medzi používateľmi alebo komentáre k plánom, by som mohol pridať nové kolekcie alebo rozšíriť existujúce dokumenty. Flexibilita NoSQL databázy umožňuje pridať nové polia do dokumentov bez nutnosti migrácie existujúcich dát, čo je výrazná výhoda oproti relačným databázam, kde by zmena schémy vyžadovala komplexnú migráciu. [9]

8 POPIS A TESTOVANIE KVALITY APLIKÁCIE

Kvalita mobilnej aplikácie sa neposudzuje len podľa funkčnosti, ale aj podľa nefunkčných aspektov, ktoré priamo ovplyvňujú používateľský zážitok. Podľa systematického prehľadu výskumu v oblasti optimalizácie výkonu mobilných aplikácií existujú štyri kľúčové metriky, ktoré určujú úspech aplikácie z pohľadu používateľa: rýchlosť odozvy, čas spustenia, spotreba pamäte a spotreba energie. [10] Pri testovaní aplikácie Travel Planner som sa preto zamerial na všetky tieto aspekty a implementoval som komplexný systém hodnotenia kvality.

8.1 DEFINÍCIA KĽÚČOVÝCH METRÍK KVALITY

Prvým krokom pri testovaní kvality bolo definovanie merateľných kritérií, ktoré presne odzrkadľujú efektívnosť a responzivnosť aplikácie. Rýchlosť odozvy predstavuje čas medzi používateľskou akciou a viditeľnou reakciou aplikácie. V kontexte Travel Planner to zahŕňa čas načítania obrazovky po prihlásení, rýchlosť navigácie medzi sekciami, čas generovania cestovného plánu a plynulosť scrollovania v zoznamoch plánov. Podľa výskumu Shboula a kol. je rýchlosť odozvy jednou z najdôležitejších metrík, ktorá priamo súvisí s používateľskou spokojnosťou a mierou odinštalovania aplikácie. [10]

Čas spustenia aplikácie (launch time) meriam od momentu, keď používateľ stlačí ikonu aplikácie, až po okamih, keď je hlavná obrazovka plne načítaná a interaktívna. Táto metrika je obzvlášť dôležitá pre prvý dojem používateľa, pretože pomalé spustenie môže viesť k negatívnemu vnímaniu aplikácie ešte predtým, ako používateľ skutočne začne aplikáciu používať. V mojej aplikácii som implementoval optimalizáciu spustenia prostredníctvom lazy loadingu komponentov a minimalizácie synchronných operácií počas inicializácie.

Spotreba pamäte je kritická metrika v prostredí mobilných zariadení, ktoré majú obmedzené hardvérové zdroje. Monitorujem celkovú spotrebu pamäte aplikácie, vrátane pamäte používanej na ukladanie obrázkov, cache dát a aktívnych view modelov. Vysoká spotreba pamäte môže viesť k pomalšiemu výkonu, častejšiemu ukončovaniu aplikácie systémom a negatívnemu dopadu na spotrebu batérie.

Spotreba energie je poslednou kľúčovou metrikou, ktorú som hodnotil. Mobilné zariadenia majú obmedzenú kapacitu batérie, a preto je efektívne využívanie energie kritické pre dlhodobú používateľskú spokojnosť. Podľa výskumu je spotreba energie

jednou z hlavných príčin odinštalovania aplikácií, pričom používatelia často kritizujú aplikácie, ktoré rýchlo vybíjajú batériu. [10] V mojej aplikácii som sa zamerial na optimalizáciu sieťových požiadaviek, efektívne využívanie cache a minimalizáciu nepotrebných pozadových operácií.

8.2 METODOLÓGIA TESTOVANIA VÝKONU

Pre hodnotenie týchto metrík som implementoval viacúrovňový prístup k testovaniu, ktorý kombinuje automatizované nástroje, manuálne testovanie a monitorovanie v produkčnom prostredí. Benchmarking zahŕňa porovnanie výkonu optimalizovanej verzie aplikácie s referenčnou verzou za štandardizovaných podmienok. Tento prístup mi umožnil vyhodnotiť efektívnosť optimalizačných stratégií a identifikovať oblasti pre ďalšie zlepšenia. [10]

Profilovanie aplikácie som vykonával pomocou nástrojov poskytovaných Xcode, konkrétne Instruments, ktoré umožňujú detailnú analýzu využívania zdrojov a výkonnostných úzkych miest. Použil som Time Profiler na identifikáciu pomalých častí kódu, Allocations instrument na monitorovanie spotreby pamäte a Energy Log na meranie spotreby energie. Tieto nástroje mi poskytli presné údaje o tom, kde aplikácia stráca výkon a kde je možné implementovať optimalizácie.

Real-world testovanie som vykonal na rôznych zariadeniach s rôznymi hardvérovými špecifikáciami, veľkosťami obrazoviek a verziami iOS. Tento prístup je nevyhnutný, pretože výkon aplikácie závisí nielen od kódu, ale aj od hardvéru, softvéru a aktuálneho prostredia vykonávania. [10] Testoval som aplikáciu na zariadeniach s rôznymi kapacitami RAM, rôznymi procesormi a rôznymi verziami iOS, aby som zabezpečil konzistentný výkon naprieč celým spektrom podporovaných zariadení.

8.3 TESTOVANIE PRESNOSTI AI

Okrem nefunkčných metrík som testoval aj funkčnosť aplikácie, konkrétne presnosť a relevantnosť odporúčaní generovaných umelou inteligenciou. Toto testovanie zahŕňalo vytvorenie testovacej sady rôznych vstupných parametrov (destinácie, rozpočty, záujmy) a následné porovnanie generovaných plánov s očakávanými výsledkami. Hodnotil som, či AI správne interpretuje požiadavky používateľa, či generuje realistické plány a či sú odporúčané aktivity a reštaurácie relevantné pre zadanú destináciu.

Pre testovanie presnosti som vytvoril sadu referenčných scenárov, kde som pre každý scenár definoval očakávané charakteristiky plánu. Napríklad pre destináciu s nízkym rozpočtom som očakával, že plán bude obsahovať bezplatné alebo lacné aktivity, zatiaľ čo pre destináciu s vysokým rozpočtom som očakával prémiové zážitky. Porovnával som generované plány s týmito očakávaniami a meral som zhody.

8.4 TESTOVANIE STABILITY

Stabilita aplikácie je ďalším kritickým aspektom kvality. Testoval som aplikáciu na rôznych scenároch použitia, vrátane extrémnych prípadov, ako sú slabé sieťové pripojenie, prerušenie sieťového pripojenia počas generovania plánu alebo súčasné vytvorenie viacerých plánov. Cieľom bolo identifikovať a opraviť potenciálne pády aplikácie, memory leaky a race conditions, ktoré by mohli viesť k nestabilite.

Používateľská prístupnosť a dizajn som hodnotil prostredníctvom testovania s reálnymi používateľmi. Tieto testy zahŕňali pozorovanie používateľov pri vykonávaní kľúčových úloh, ako je vytvorenie prvého plánu, navigácia medzi plánmi a úprava existujúceho plánu. Zhromažďoval som spätnú väzbu o intuitívnosti rozhrania, jasnosti navigácie a celkovom používateľskom zážitku. Táto spätná väzba mi umožnila identifikovať oblasti, kde dizajn alebo funkčnosť mohli byť zlepšené.

8.5 KONTINUÁLNE MONITOROVANIE A OPTIMALIZÁCIA

Testovanie kvality nie je jednorazová aktivita, ale kontinuálny proces, ktorý pokračuje aj po nasadení aplikácie do produkcie. Implementoval som systém monitorovania, ktorý zhromažďuje anonymizované údaje o výkone aplikácie od reálnych používateľov. Tento systém sleduje kľúčové metriky, ako sú čas spustenia, frekvencia pádu aplikácie, spotreba pamäte a čas odozvy na používateľské akcie.

Tieto údaje mi umožňujú identifikovať problémy s výkonom, ktoré sa môžu vyskytnúť len v produkčnom prostredí alebo na konkrétnych zariadeniach, a následne implementovať ciele optimalizácie. Podľa výskumu je kontinuálne monitorovanie a optimalizácia kľúčové pre dlhodobý úspech mobilnej aplikácie, pretože používateľské potreby, vzory použitia a technologické prostredie sa neustále vyvíjajú. [10]

9 STRATÉGIA NASADENIA APLIKÁCIE DO APP STORE

Nasadenie aplikácie do App Store je komplexný proces, ktorý vyžaduje dôkladnú prípravu, dodržanie prísnych smerníc a strategické plánovanie. App Store predstavuje prostredie, kde každá aplikácia prechádza expertným hodnotením, čo zabezpečuje bezpečnosť a kvalitu pre miliardy používateľov. [11]

Pre úspešné nasadenie aplikácie Travel Planner som vypracoval komplexnú stratégiu, ktorá pokrýva všetky aspekty od technickej prípravy až po marketingovú prezentáciu.

9.1 PRÍPRAVA APLIKÁCIE NA KONTROLU

Pred samotným odoslaním aplikácie na review som vykonal dôkladnú kontrolu všetkých požiadaviek, ktoré App Store kladie na aplikácie. Prvým krokom bolo zabezpečenie, že aplikácia je kompletná a plne funkčná. Podľa App Store Review Guidelines musia byť všetky submisie finálnymi verziami s kompletnými metadátami a plne funkčnými URL adresami. Placeholder texty, prázdne webové stránky a dočasný obsah musia byť pred submisiou odstránené. [11]

Kritickým aspektom prípravy bolo testovanie aplikácie na skutočných zariadeniach na prítomnosť chýb a nestability. Aplikácia musí byť otestovaná na rôznych zariadeniach s rôznymi hardvérovými špecifikáciami a verziami iOS, aby som zabezpečil, že funguje konzistentne naprieč celým spektrom podporovaných zariadení. Pre aplikácie s prihlasovaním, ako je Travel Planner, som musel zabezpečiť, že backend služby sú aktívne a dostupné počas review procesu, a poskytnúť demo účet alebo plne funkčný demo mód pre App Review tím. [11]

Ďalším dôležitým krokom bolo zabezpečenie, že aplikácia dodržiava všetky bezpečnostné a právne požiadavky. Aplikácia musí implementovať vhodné bezpečnostné opatrenia na správne spracovanie používateľských informácií a zabránenie neoprávnenému prístupu tretích strán. [11] V mojej aplikácii som implementoval Firebase Security Rules, ktoré zabezpečujú, že používateľské dáta sú chránené a prístupné len autorizovaným používateľom. Zároveň som zabezpečil, že aplikácia má kompletnú a presnú politiku ochrany súkromia, ktorá jasne popisuje, aké dáta aplikácia zhromažďuje, ako ich používa a ako ich uchováva.

9.2 METADATA A DOKUMENTÁCIA

Kvalitné metadata sú kľúčové pre úspešné schválenie aplikácie a jej následnú viditeľnosť v App Store. Všetky informácie o aplikácii, vrátane popisu, snímok obrazovky a preview videí, musia presne odzrkadľovať skutočnú funkcionality aplikácie. [11] Pre Travel Planner som pripravil detailný popis aplikácie, ktorý jasne vysvetľuje hlavné funkcie a výhody aplikácie, bez zavádzajúcich tvrdení alebo nereálnych očakávaní.

Snímky obrazovky musia zobrazovať aplikáciu v skutočnom použití, nie len titulnú grafiku, prihlasovaciu stránku alebo úvodnú obrazovku. [11] Pre moju aplikáciu som pripravil sériu snímok obrazovky, ktoré demonštrujú kľúčové funkcie: vytvorenie cestovného plánu, zobrazenie detailov plánu, navigáciu medzi plánmi a onboarding proces. Tieto snímky poskytujú používateľom jasnú predstavu o tom, čo môžu očakávať od aplikácie.

Preview video je ďalším dôležitým nástrojom na prezentáciu aplikácie. Previews môžu obsahovať len video záznamy obrazovky samotnej aplikácie, s možnosťou pridať komentár alebo textové preklady na vysvetlenie funkcií, ktoré nie sú z videa jasné. [11] Pre Travel Planner som vytvoril krátke video, ktoré demonštruje celý proces od prihlásenia cez vytvorenie plánu až po zobrazenie finálneho itinerára. Kategória aplikácie musí byť vybraná presne podľa skutočnej funkcionality. [11] Travel Planner som zaradil do kategórie "Travel", čo najlepšie vystihuje účel aplikácie. Vekové hodnotenie som nastavil podľa obsahu aplikácie, pričom som dôsledne zodpovedal všetky otázky týkajúce sa vekového hodnotenia v App Store Connect.

9.3 BEZPEČNOSTNÉ A PRÁVNE ASPEKTY

App Store kladie veľký dôraz na bezpečnosť a ochranu súkromia používateľov. Aplikácia musí mať kompletnú politiku ochrany súkromia, ktorá je dostupná v App Store Connect metadatach aj priamo v aplikácii na ľahko dostupnom mieste. [11] Politika ochrany súkromia musí jasne a explicitne identifikovať, aké dáta aplikácia zhromažďuje, ako ich zhromažďuje a na aké účely ich používa. Musí tiež potvrdiť, že akákoľvek tretia strana, s ktorou aplikácia zdieľa používateľské dáta, poskytuje rovnakú alebo rovnocennú ochranu dát, ako je uvedené v politike ochrany súkromia aplikácie.

Aplikácie, ktoré zhromažďujú používateľské alebo používateľské dáta, musia získať súhlas používateľa so zhromažďovaním, aj keď sa tieto dáta v čase

zhromažďovania považujú za anonymné. [11] V Travel Planner som implementoval jasné dialógy na získanie súhlasu, ktoré používateľovi vysvetľujú, prečo aplikácia potrebuje prístup k určitým dátam, ako sú informácie o lokalite alebo kontaktné údaje. Aplikácia musí poskytnúť používateľovi ľahko dostupný a zrozumiteľný spôsob, ako môže súhlas odvolať.

Pre aplikácie, ktoré podporujú vytváranie účtov, je povinné poskytnúť možnosť vymazania účtu priamo v aplikácii. [11] V Travel Planner som implementoval funkciu na vymazanie účtu, ktorá umožňuje používateľovi trvalo odstrániť všetky svoje dáta z databázy. Táto funkcia je dôležitá nielen pre dodržanie App Store smerníc, ale aj pre dodržanie právnych požiadaviek, ako je GDPR.

9.4 PROCES REVIEW A KOMUNIKÁCIA

Po odoslaní aplikácie na review začal proces hodnotenia, ktorý môže trvať od niekoľkých hodín až po niekoľko dní, v závislosti od zložitosti aplikácie a prítomnosti nových problémov. [11] App Review tím skúma aplikáciu čo najskôr, ale komplexnejšie aplikácie alebo aplikácie s novými problémami môžu vyžadovať väčšiu pozornosť a zváženie.

Počas review procesu som sledoval stav aplikácie v App Store Connect, kde sa zobrazujú aktuálne informácie o stave submisie. [11] V prípade, že by aplikácia bola zamietnutá, App Store Connect poskytuje možnosť priamej komunikácie s App Review tímom, kde môžem poskytnúť dodatočné informácie alebo vysvetliť konkrétne funkcie aplikácie. Táto komunikácia môže pomôcť aplikácii dostať sa do obchodu a zároveň pomôcť zlepšiť App Review proces. V prípade kritických časových problémov existuje možnosť požiadať o urýchléné review. [11] Túto možnosť som však plánoval využiť len v skutočne nevyhnutných prípadoch, pretože zneužívanie tohto systému môže viesť k zamietnutiu budúcich žiadostí o urýchlénie. Po schválení aplikácie App Review tímom a nasadení do App Store je dôležité pokračovať v monitorovaní a údržbe aplikácie. Aplikácia musí zostať funkčná a aktualizovaná, aby spĺňala očakávania existujúcich zákazníkov. [11] Aplikácie, ktoré prestanú fungovať, môžu byť z App Store kedykoľvek odstránené. Pre budúce aktualizácie som pripravil stratégiu, ktorá zahŕňa pravidelné kontroly funkčnosti aplikácie, monitorovanie používateľských recenzií a spätnej väzby, a implementáciu vylepšení na základe používateľských potrieb.

10 NÁVRH MARKETINGOVEJ STRATÉGIE

Úspešné uvedenie aplikácie do App Store vyžaduje nielen technickú prípravu, ale aj dôkladne premyslenú marketingovú a predajnú stratégiu. App Store predstavuje globálny trh s viac ako miliardou používateľov a viac ako pol miliardou týždenných návštevníkov v 175 krajinách, čo poskytuje obrovské príležitosti pre malých vývojárov, ktorí dokážu efektívne prezentovať svoju aplikáciu. [12] Pre aplikáciu Travel Planner som vypracoval komplexnú marketingovú stratégiu, ktorá využíva silné stránky App Store ekosystému a ciele na dosiahnutie udržateľného rastu a úspechu.

10.1 APP STORE OPTIMIZATION (ASO) STRATÉGIA

App Store Optimization je základom marketingovej stratégie pre každú mobilnú aplikáciu. ASO zahŕňa optimalizáciu všetkých prvkov, ktoré ovplyvňujú viditeľnosť a konverziu aplikácie v App Store. Prvým krokom je výber správnej kategórie aplikácie. Travel Planner som zaradil do kategórie "Travel", čo najlepšie vystihuje účel aplikácie a umožňuje mi konkurovať v relevantnej kategórii.

Názov aplikácie a podnázov sú kritické prvky ASO, pretože ovplyvňujú vyhľadávanie a prvý dojem používateľov. Názov aplikácie musí byť jedinečný, zapamätateľný a obsahovať kľúčové slová, ktoré používatelia hľadajú. Pre Travel Planner som zvolil názov, ktorý jasne komunikuje účel aplikácie a zároveň obsahuje relevantné kľúčové slová pre vyhľadávanie. Podnázov poskytuje ďalšiu príležitosť na komunikáciu hlavných funkcií aplikácie a prilákanie pozornosti používateľov.

Kľúčové slová sú ďalším dôležitým prvkom ASO. Musím vybrať kľúčové slová, ktoré presne opisujú funkcionality aplikácie a zároveň sú relevantné pre vyhľadávanie používateľov. Pre Travel Planner som identifikoval kľúčové slová ako "travel planning", "itinerary", "trip planner", "vacation planner" a podobné termíny, ktoré používatelia pravdepodobne použijú pri hľadaní aplikácie na plánovanie ciest.

Snímky obrazovky a preview video sú kľúčové pre konverziu používateľov z návštevníkov na sťahovateľov. Podľa výskumu App Store ekosystému majú aplikácie s kvalitnými vizuálnymi materiálmi výrazne vyššiu mieru konverzie. [12] Pre Travel Planner som pripravil sériu snímok obrazovky, ktoré demonštrujú kľúčové funkcie aplikácie: vytvorenie plánu pomocou AI, zobrazenie detailného plánu, navigáciu medzi plánmi a onboarding proces. Každá snímka obsahuje krátky popisný text, ktorý vysvetľuje výhodu zobrazenej funkcie.

10.2 GLOBÁLNA EXPANZIA A LOKALIZÁCIA

App Store poskytuje jedinečnú príležitosť pre globálnu distribúciu aplikácie bez potreby vytvárať vlastnú infraštruktúru pre každú krajinu. Podľa výskumu App Store ekosystému pochádza približne 40% všetkých sťahovaní aplikácií od malých vývojárov z používateľov mimo domovskej krajiny vývojára. [12] Táto štatistika dokazuje, že globálna expanzia je kľúčová pre úspech aplikácie, najmä pre aplikácie s univerzálnym zámerom, ako je Travel Planner.

Pre úspešnú globálnu expanziu som vypracoval stratégiu lokalizácie aplikácie. Prvým krokom je identifikácia kľúčových trhov na základe analýzy konkurencie, veľkosti trhu a potenciálu pre aplikácie na plánovanie ciest. Pre Travel Planner som identifikoval primárne trhy: Spojené štáty, Veľká Británia, Nemecko, Francúzsko, Austrália a ďalšie anglicky hovoriace krajiny, kde je cestovanie populárne a používatelia majú vysokú kúpnu silu. Lokalizácia aplikácie zahŕňa nielen preklad textu, ale aj prispôbenie obsahu lokálnym zvykom a kultúre. Pre Travel Planner to znamená, že AI generované plány musia brať do úvahy lokálne špecifiká, ako sú otváracie hodiny, miestne zvyky a kultúrne preferencie. Zároveň som pripravil lokalizované snímky obrazovky a popisy aplikácie pre každý kľúčový trh, aby som maximalizoval relevanciu a príťažlivosť pre lokálnych používateľov.

10.3 CENOVÁ STRATÉGIA A MONETIZÁCIA

Výber správnej cenovej stratégie je kritický pre úspech aplikácie. App Store poskytuje flexibilné možnosti monetizácie, vrátane jednorazových platieb, predplatného a in-app purchases. Pre Travel Planner som zvolil model freemium s predplatným, kde základné funkcie sú dostupné zdarma, ale pokročilé funkcie, ako sú neobmedzené plány, export do kalendára a offline prístup, vyžadujú predplatné. Táto stratégia je založená na úspešných príkladoch aplikácií v App Store ekosystéme, kde aplikácie s predplatným dosahujú vyššiu mieru retencie a dlhodobú hodnotu používateľa. [12] Model freemium umožňuje používateľom vyskúšať aplikáciu bez rizika, čo zvyšuje počet sťahovaní a poskytuje príležitosť na konverziu na platených používateľov prostredníctvom kvalitného používateľského zážitku.

10.4 MARKETING A PROPAGÁCIA

Okrem ASO som vypracoval stratégiu pre marketingové kanály mimo App Store. Sociálne médiá sú kľúčové pre budovanie komunity a zvyšovanie povedomia o aplikácii. Pre Travel Planner som identifikoval relevantné platformy, ako sú Instagram, TikTok a Pinterest, kde môžem zdieľať vizuálne príťažlivý obsah, ako sú screenshots z aplikácie, príklady generovaných plánov a používateľské príbehy.

Obsahový marketing je ďalším dôležitým bodom. Vytvoril som stratégiu pre blogové príspevky a články o cestovaní, ktoré poskytujú hodnotu používateľom a zároveň predstavujú aplikáciu ako riešenie ich problémov. Tieto články môžu byť zdieľané na sociálnych sieťach a pomôcť zlepšiť SEO a organický dosah aplikácie.

Partnerships a spolupráca s influencerami v oblasti cestovania sú ďalšou súčasťou marketingovej stratégie. Identifikoval som relevantných influencerov a cestovateľské blogy, s ktorými môžem spolupracovať na propagácii aplikácie. Tieto spolupráce môžu poskytnúť autentickú spätnú väzbu a zvyšovať dôveryhodnosť aplikácie medzi cieľovou skupinou.

10.5 BUDOVANIE KOMUNITY

Dlhodobý úspech aplikácie závisí nielen od získavania nových používateľov, ale aj od udržania existujúcich používateľov. Podľa výskumu App Store ekosystému dosahujú aplikácie s vysokou mierou retencie lepšie výsledky a vyššiu mieru monetizácie. [12] Pre Travel Planner som vypracoval retenčnú stratégiu, ktorá zahŕňa pravidelné aktualizácie, nové funkcie a angažovanosť s komunitou. Taktiež som navrhol notifikácie, ktoré poskytujú hodnotu, ako sú pripomienky na nadchádzajúce cesty, tipy na plánovanie a aktualizácie o nových funkciách. Pravidelné aktualizácie aplikácie sú kľúčové pre udržanie používateľov a zlepšovanie hodnotenia aplikácie. Plánujem pravidelné aktualizácie, ktoré pridávajú nové funkcie, zlepšujú výkon a opravujú chyby.

11 ZÁVER

Cieľom mojej maturitnej práce bolo navrhnuť a implementovať funkčnú mobilnú aplikáciu AI Travel Planner, ktorá využíva moderné technológie umelej inteligencie na zefektívnenie procesu plánovania cestovania. V dobe informačného presýtenia predstavuje tento nástroj riešenie, ktoré šetrí čas používateľa a poskytuje mu personalizovaný zážitok v priebehu niekoľkých sekúnd.

V teoretickej a realizačnej časti som podrobne dokumentoval kľúčové aspekty vývoja. Rozhodnutie pre natívnu platformu iOS a programovací jazyk Swift sa na základe vykonaných analýz výkonu ukázalo ako opodstatnené, najmä z hľadiska plynulosti používateľského rozhrania a efektívnej správy systémových prostriedkov. Implementácia architektúry MVVM v kombinácii s frameworkom SwiftUI zabezpečila čistý, modulárny a ľahko udržiavateľný kód.

Z hľadiska bezpečnosti a infraštruktúry považujem za kľúčové úspešné nasadenie modelu Backend-for-Frontend (BFF) na platforme Firebase. Tento prístup mi umožnil nielen bezpečne spravovať API kľúče k službe OpenAI, ale aj zabezpečiť robustnú autentifikáciu používateľov a škálovateľné ukladanie dát v NoSQL databáze.

Práca na tomto projekte mi priniesla cenné skúsenosti v oblastiach, ktoré dnes formujú technologický priemysel, či už od Prompt Engineeringu a integrácie LLM modelov až po pokročilý mobilný vývoj a cloudovú architektúru. Hoci je aplikácia v súčasnom stave plne funkčná, vnímam priestor pre jej ďalšie rozširovanie, napríklad o integráciu interaktívnych máp, zdieľanie plánov medzi používateľmi či offline režim.

Záverom možno konštatovať, že stanovené ciele práce boli splnené. Vytvorená aplikácia nie je len technickým demonštrátorom možností umelej inteligencie, ale praktickým nástrojom, ktorý má potenciál reálne uľahčiť život modernému cestovateľovi.

12 ZOZNAM POUŽITEJ LITERATÚRY

[1] CHEN, Eric, et al. OAuth Demystified for Mobile Application Developers. In: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security. Scottsdale, Arizona, USA [cit. 12-13-2025] Dostupné na internete: <<https://www.microsoft.com/en-us/research/wpcontent/uploads/2016/02/OAuthDemystified.pdf>>

[2] BOYRAZ, Uğur Can. A Comparative Study of Performance and Resource Efficiency Between Flutter and Native Android Applications. Tallinn: Estonian Entrepreneurship University of Applied Sciences, 2025. [cit. 12-13-2025] Dostupné na internete: <<https://eek.ee/download.php?t=kb&dok=p1ja9nfggn1cej14tbp801cuc1gqf3.pdf>>

[3] JUNG, Hyunwoo a Nari SEO. Modern iOS App Development with SwiftUI and Combine: A Declarative Approach to UI and State. American Journal of Engineering, Mechanics and Architecture. 2023, [cit. 12-13-2025] Dostupné na internete: <<http://eprints.umsida.ac.id/16183/1/113-123%2BModern%2BBIOS%2BApp%2BDevelopment%2Bwith%2BSwiftUI%2Band%2BCombine.pdf>>

[4] OPENAI. Prompt engineering guide [online]. 2024 [cit. 12-13-2025] Dostupné na internete: <<https://www.ki-insights.com/wp-content/uploads/2024/02/Prompt-engineering-OpenAI-API.pdf>>

[5] MOLOUDI, Behzad. Optimizing Architecture in iOS Development 2025. [cit. 1-2-2026] Dostupné na internete: <https://www.theseus.fi/bitstream/handle/10024/908051/Moloudi_Behzad.pdf;jsessionid=685BCBABEBD170E9757B8F6BE5D4DE76?sequence=2>

[6] GRIFFITHS, Stephen. Mobile App UX Principles 2015 [cit. 1-2-2026] Dostupné na internete: <https://www.thinkwithgoogle.com/_qs/documents/2081/Mobile_App_UX_Principles_3.pdf>

- [7] LAPALME, Guy. Some SwiftUI fundamentals. Montréal: Université de Montréal [cit. 1-2-2026] Dostupné na internete: <<http://www.iro.umontreal.ca/~lapalme/SwiftUI-Fundamentals/SwiftUI%20Fundamentals.pdf>>
- [8] GOOGLE. *Firebase Security Rules* [online]. 2026 [cit. 1-2-2026]. Dostupné z: <<https://firebase.google.com/docs/rules>>
- [9] MIOR, Michael J., et al. NoSE: Schema Design for NoSQL Applications. IEEE Transactions on Knowledge and Data Engineering, 2016. [cit. 1-2-2026]. Dostupné z: <<https://ashraf.aboulnaga.me/pubs/tkde17nose.pdf>>
- [10] SHBOUL, Amer Qasem. Performance Optimization of Mobile Apps. 2020. [cit. 1-2-2026]. Dostupné z: <<https://sjr-publishing.com/wp-content/uploads/2019/03/Performance-Optimization-of-Mobile-Apps-2.pdf>>
- [11] APPLE INC. App Review Guidelines. Cupertino: Apple Inc., 2025. [cit. 1-2-2026]. Dostupné z: <<https://developer.apple.com/support/downloads/terms/app-review-guidelines/App-Review-Guidelines-20250609-English-UK.pdf>>
- [12] BORCK, Jonathan, CAMINADE, Juliette, VON WARTBURG, Markus. A Global Perspective on the Apple App Store Ecosystem: An exploration of small businesses within the App Store ecosystem. June 2020. [cit. 1-2-2026]. Dostupné z : <<https://www.apple.com/newsroom/pdfs/apple-app-store-study-2020.pdf>>

PRÍLOHA A – USB KĽÚČ

Priložené USB obsahuje digitálnu verziu odbornej práce (PDF) a zdrojový kód vyvinutej aplikácie.